

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»

Институт информационных технологий

Кафедра микропроцессорных систем и сетей

А. А. Москалев, Н. И. Криштапович

АРХИТЕКТУРА ОПЕРАЦИОННЫХ СИСТЕМ

Лабораторный практикум
для слушателей переподготовки по специальности 1-40 01 73
«Программное обеспечение информационных систем»
всех форм обучения

В 4-х частях

Часть 1

Интерфейс пользователя в консольном режиме

Минск БГУИР 2011

УДК 004.451(076.5)
ББК 32.973.202я73
М82

Р е ц е н з е н т ы:

заведующий кафедрой менеджмента Минского института управления,
доктор технических наук, профессор В. А. Вишняков;

заведующий кафедрой программного обеспечения информационных технологий
учреждения образования «Белорусский государственный университет
информатики и радиоэлектроники», кандидат технических наук,
доцент В. В. Бахтизин

Москалев, А. А.

М82 Архитектура операционных систем : лаб. практикум для слушателей
переподготовки по спец. 1-40 01 73 «Программное обеспечение инфор-
мационных систем» всех форм обуч. В 4 ч. Ч. 1: Интерфейс пользователя
в консольном режиме / А. А. Москалев, Н. И. Криштапович. – Минск :
БГУИР, 2011. – 24 с.: ил.

ISBN 978-985-488-718-0 (ч. 1).

Рассмотрена организация интерфейса пользователя в консольном режиме OS Windows различных версий. Пособие может быть полезно как слушателям, выполняющим лабораторный практикум, так и преподавателям, осуществляющим руководство выполнением лабораторных работ по курсу «Архитектура операционных систем».

**УДК 004.451(076.5)
ББК 32.973.202я73**

**ISBN 978-985-488-718-0 (ч. 1)
ISBN 978-985-488-719-7**

© Москалев А. А., Криштапович Н. И., 2011
© УО «Белорусский государственный
университет информатики
и радиоэлектроники», 2011

Содержание

Лабораторная работа «Интерфейс пользователя в консольном режиме»	4
1 Цель работы.....	4
2 Теоретические сведения.....	4
2.1 Командная оболочка.....	4
2.2 Запуск и настройка командной строки.....	5
2.3 Структура и синтаксис команд	6
2.3.1 Структура команды.....	6
2.3.2 Получение справки и соглашения по описанию синтаксиса...	6
2.3.3 Внешние и внутренние команды	8
2.4 Имена файлов в качестве параметров.....	8
2.4.1 Полное имя файла.....	8
2.4.2 Абсолютный и относительный путь, текущая директория.....	8
2.4.3 Длинные имена файлов в командной строке	8
2.4.4 Специальные файлы.....	9
2.5 Несколько команд в одной строке и условное выполнение	10
2.6 Использование переменных среды в Cmd.exe	11
2.6.1 Блок среды командной оболочки.....	11
2.6.2 Локализация изменений в блоке среды.....	12
2.6.3 Установка значений переменных среды.....	12
2.6.4 Обращение к значениям переменных среды.....	14
2.6.5 Переменные среды для Windows XP.....	15
2.7 Операторы перенаправления ввода–вывода команд.....	16
2.8 Команды-фильтры и конвейер команд.....	18
3 Выполнение работы.....	19
3.1 Подготовка к занятию.	19
3.2 Запуск и настройка окна консоли.	19
3.3 Знакомство с отдельными командами	20
3.3.1 Просмотр директорий и свойств файлов. Команда DIR.....	20
3.3.2 Создание директорий	20
3.3.3 Копирование и просмотр файлов.	21
3.3.4 Перенаправление ввода–вывода.....	21
3.3.5 Условное выполнение команд в строке.....	21
3.3.6 Использование значений переменных среды	22
4 Вопросы для самопроверки	22
Литература.....	23

Лабораторная работа

«Интерфейс пользователя в консольном режиме»

1 *Цель работы*

Изучить организацию интерфейса пользователя в режиме консоли Windows, научиться применять команды пользовательского интерфейса на практике.

2 *Теоретические сведения*

2.1 *Командная оболочка*

Командная оболочка (консоль, или командная строка) – это отдельная программа, которая обеспечивает прямую связь между пользователем и операционной системой.

Пользовательский интерфейс командной строки представляет собой среду, в которой приложения и служебные программы осуществляют ввод–вывод в текстовом режиме. Вводимые и выводимые данные отображаются в консольном окне, которое похоже на экран MS DOS. Управление системой осуществляется путем ввода текстовых команд. Вводимые в интерактивном режиме команды воспринимаются и интерпретируются программой консоли. Поэтому у программы консоли есть еще одно название – командный процессор. Для MS DOS и Windows 95/98 командным процессором является программа **Command.com**. Для Windows NT и ее производных функции консоли обеспечивает программа **Cmd.exe**. Командный процессор **Cmd**, несмотря на сходство внешнего представления консольного окна, поддерживает значительно более широкий спектр команд и возможностей по сравнению с **Command.com**.

Настоящий материал подготовлен на базе справочника [1] в качестве справочно-теоретического раздела лабораторной работы. Однако авторы постарались учесть постоянное развитие операционной среды, собственный практический опыт, обнаруженные опечатки, и внесли соответствующие изменения в излагаемый материал.

Пользовательский интерфейс командной строки представляет собой среду, в которой приложения и служебные программы осуществляют ввод–вывод в текстовом режиме. Вводимые и выводимые данные отображаются в консольном окне. Для создания консольного окна необходимо запустить одну из указанных выше программ. В дальнейшем для определенности будем рассматривать возможности и свойства консоли для Windows XP (программа **Cmd.exe**).

2.2 Запуск и настройка командной строки

В Windows XP файл **Cmd.exe** находится в каталоге, который обозначен в реестре как **%SystemRoot%\SYSTEM32**. Здесь **%SystemRoot%** является значением переменной **SystemRoot** блока окружения (среды). Значение этой переменной указывает на системный каталог Windows, обычно это **C:\Windows**.

Для запуска командного процессора (открытия нового сеанса командной строки) можно выбрать пункт **Выполнить...** (**Run**) в меню **Пуск (Start)**, ввести имя файла **Cmd.exe** и нажать кнопку **ОК**. В результате откроется новое окно (рисунок 1), в котором можно запускать команды и видеть результат их работы.

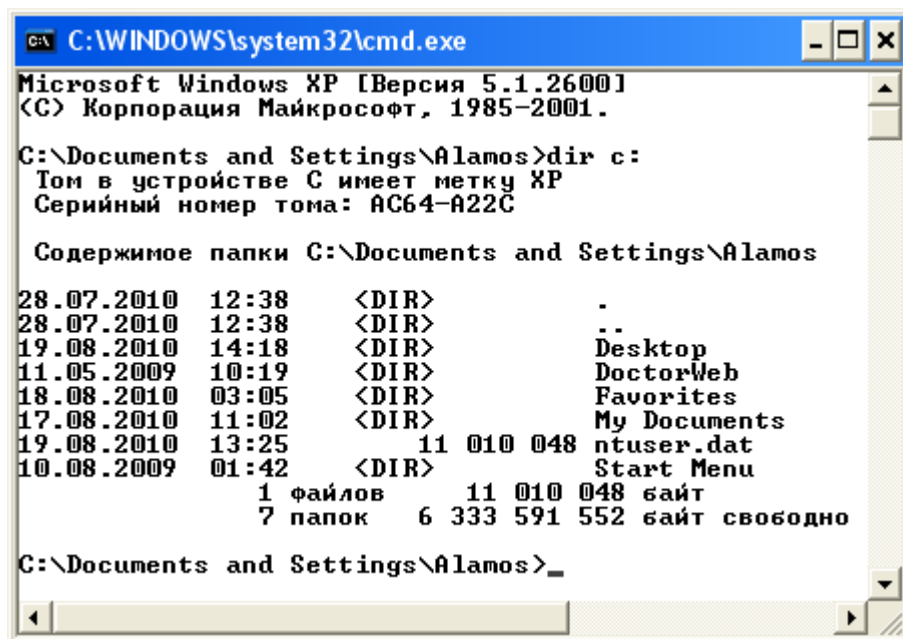


Рисунок 1 – Пример консольного окна в Windows XP

В консольном окне вывод осуществляется в текстовом режиме. Это означает, что каждый символ изображается в пределах знакоместа определенного размера. В строке стандартной (как в MS DOS) консоли располагается 80 знакомест. Такая консоль используется в Windows 95/98. Для консоли Windows NT/2000/XP ширину строки можно изменять в пределах от 1 до 9999 знакомест.

Количество сохраняемых и доступных для просмотра строк также задается в параметрах настройки консольного окна. Для Windows NT/2000/XP этот параметр называется «высота буфера». Максимальное значение размера буфера экрана по высоте равно 999. Такой размер позволяет выполнять прокрутку в окне командной строки и просматривать до 999 ранее выведенных строк.

Можно использовать более одного буфера. Увеличение количества буферов увеличивает число процессов, для каждого из которых сохраняется история (список ранее введенных команд).

Установка флага «Выделение мышью» позволяет копировать и вставлять данные в окне командной строки с помощью мыши. Чтобы скопировать текст,

выделите его левой кнопкой мыши в окне командной строки и нажмите правую кнопку мыши. Чтобы выполнить вставку в командную строку или текстовый файл, щелкните правой кнопкой мыши в нужном месте.

2.3 Структура и синтаксис команд

2.3.1 Структура команды

Каждая вводимая в консоли команда имеет следующую структуру:

<Имя команды> <P1> <P2> ... <Pm> /K1/K2 .../Kn,

где **<Имя команды>** – это либо имя одной из внутренних команд (например, **Copy**, **Dir**), либо полное или частичное имя файла внешней команды (например, **C:\WINDOWS\system32\debug.exe**);

<P1> <P2> ... <Pm> – параметры команды. Параметр может принимать любое значение из некоторого допустимого множества для данного параметра;

/K1/K2 .../Kn – ключи (опции, режимы) исполнения команды. Обычно ключи начинаются с символа **/**, но в некоторых командах могут начинаться с символа **-** (минус) или символа **+** (плюс). Ключи определяют режим выполнения команды. Ключи не могут принимать произвольные значения, они выбираются из predetermined для каждой команды множества.

Элементы команды отделяются друг от друга одним из разделителей. Обычно это пробел. Кроме пробела могут использоваться некоторые другие символы, назначение которых будет рассмотрено в других подразделах данного описания.

Для того чтобы выполнить команду, необходимо после получения приглашения ввести необходимые элементы команды (имя, параметры, ключи) и нажать клавишу **«Enter»**.

2.3.2 Получение справки и соглашения по описанию синтаксиса

Для получения списка доступных команд достаточно в консольном окне ввести команду **HELP** без параметров (рисунок 2). Для получения справки по отдельной команде достаточно ввести **HELP <имя команды>**. Другой вариант – ввести эту команду с ключом **«/?»**.

Ниже на рисунке 3 представлен пример запроса справки по команде **Copy**. В описании синтаксиса команды **Copy** (как и других команд) специальным образом используются некоторые символы. Так в квадратные скобки (**[...]**) заключают части команды, которые не являются обязательными и могут отсутствовать в конкретном варианте записи команды. Например, элемент **[/D]** в описании команды **Copy** говорит, что шифрование не является обязательным и ключ **«/D»** можно не указывать. Символ вертикальной черты (**|**) в описании синтаксиса применяется для разделения альтернативно выбираемых элементов команды. Например, запись **[/A|/B]** говорит о несовместимости

ключей «/A» и «/B». Слова в описании синтаксиса, например, «источник», обозначают параметры, принимающие произвольные значения.

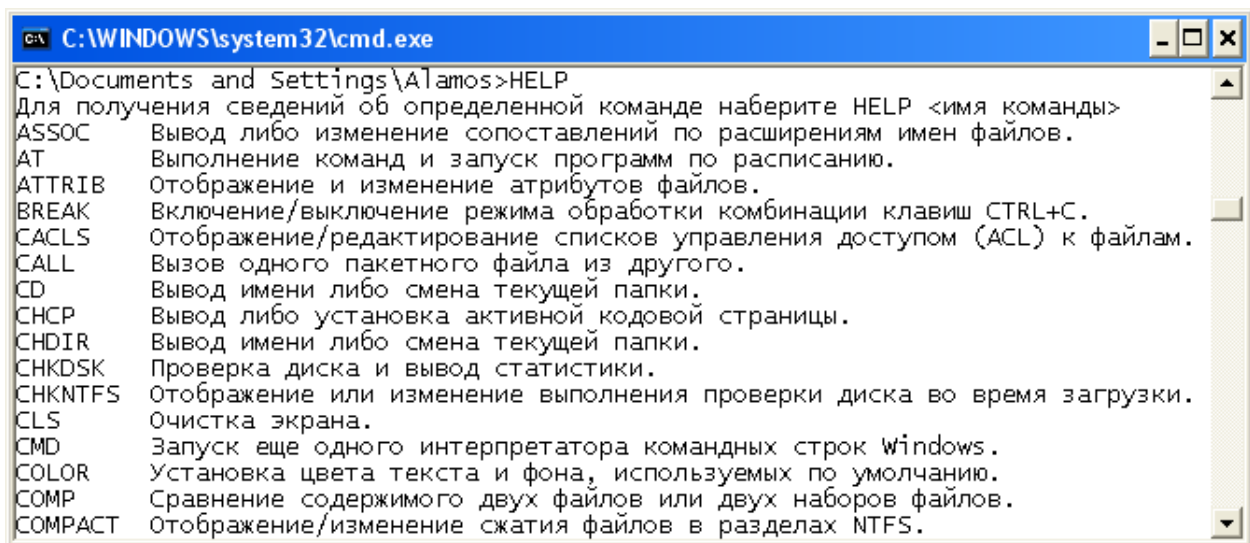


Рисунок 2 – Вид консоли после ввода команды **HELP**

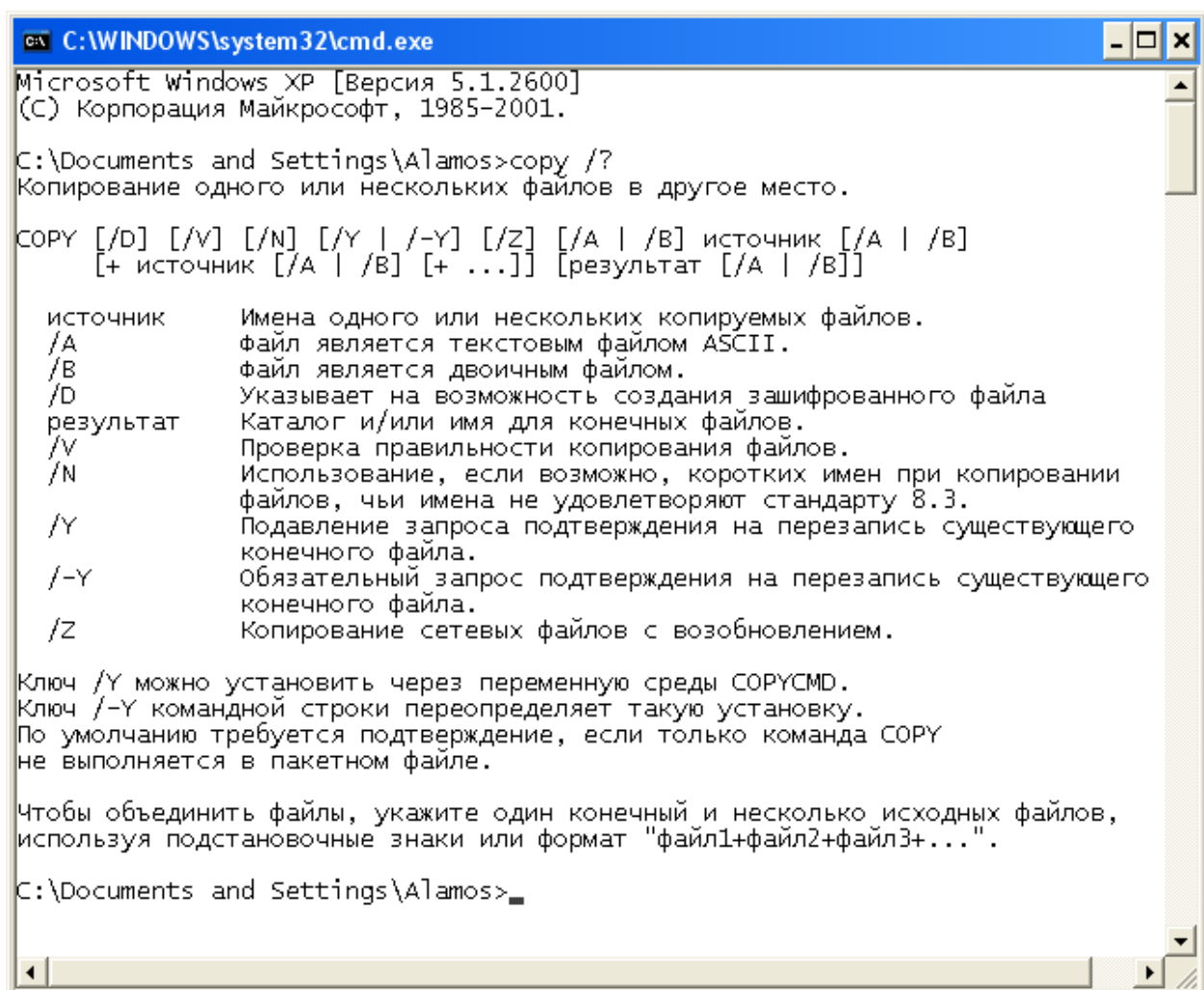


Рисунок 3 – Пример получения справочной информации о команде **Copy**

2.3.3 *Внешние и внутренние команды*

Команды, которые выполняются непосредственно самим командным интерпретатором (встроены в него), называются внутренними. Внешние команды – это отдельные программы, расположенные в виде файлов на дисках. Если не указан полный (абсолютный) путь к файлу внешней команды, то к относительному пути добавляется подстановочный путь (путь в текущую директорию). Подстановочный и относительный пути вместе образуют полный путь, который и используется для поиска файла команды. В случае отрицательного результата поиск продолжается в директориях, указанных в виде значения переменной **PATH** блока среды (environment block).

2.4 *Имена файлов в качестве параметров*

2.4.1 *Полное имя файла*

При указании файла в качестве параметра в консольной команде возможно указание полного имени файла либо его части.

Полное имя может включать следующие компоненты:

[d:] [path] filename [.ext] ,

где **d:** – устройство хранения, диск (**C:**, **D:** и т. п.);

path – путь к каталогу (**\TEMP**, **\work** и т. п.);

filename – собственно имя файла;

ext – расширение имени файла (**.txt**, **.exe**, **.COM** и т. п.).

Как видим, обязательным является только собственно имя файла.

2.4.2 *Абсолютный и относительный путь, текущая директория*

Путь может быть абсолютным или относительным.

Абсолютный путь представляет собой перечень директорий **от корня** файловой системы (в Windows символ **** в начале пути) до директории, включающей собственно имя файла.

Относительный путь указывается от текущей директории. Путь в текущую директорию называют еще подстановочным. Он дополняет относительный путь до абсолютного пути.

Текущая директория в консоли Windows – это директория, заданная командой **CHDIR (cd)**. Текущая директория в консоли определяется для каждого тома файловой системы. Для приложений Windows текущая директория определяется для каждого процесса.

2.4.3 *Длинные имена файлов в командной строке*

Собственно имя файла и расширение могут быть либо в коротком формате (8.3), либо в длинном. Длинное имя может иметь длину до 255 символов. При этом полное имя файла (включая расширение) в Windows не может содержать

более 260 символов. Данное значение определено константой `MAX_PATH` в заголовочных файлах системы программирования для Windows API.

В Windows в имени файла запрещено использование некоторых служебных символов: `\, /, :, *, ?, ", <, >, |`.

В длинном имени допускается использование пробелов. Однако поскольку пробел используется и в качестве разделителя, то для устранения возникающей неопределенности длинные имена с пробелами заключаются в кавычки. Это соглашение действует до тех пор, пока кавычки не являются частью синтаксиса команды, как это происходит, например, в командах **START** и **FOR**.

Так команда **START** предполагает, что первая переданная ей строка в кавычках является заголовком нового консольного окна. Поэтому команда

```
START "%windir%\system32\calc.exe"
```

приведет к открытию новой консоли, но не к запуску калькулятора. Правильным для запуска калькулятора будет следующее написание:

```
START "Заголовок" "%windir%\system32\calc.exe".
```

Вторым исключением из общего правила является команда **FOR**. При использовании этой команды с ключом **R** для сканирования каталога (**FOR /R ...**) имя каталога должно быть коротким именем без кавычек. Эта же команда с ключом **F** (**FOR /F ...**) воспринимает кавычки как признак непосредственно заданной текстовой строки.

В указанных и других подобных ситуациях проблему «кавычек» можно решить с помощью опции **usebackq** (начиная с Win2k), либо предварительно сохраняя командную строку в переменной блока среды.

Примечание. Юникодные версии некоторых функций позволяют использовать имена файлов, содержащие до 32 000 символов. Такие имена начинаются с префикса «`\\?\`». Например: `\\?\C:\Windows\System32\calc.exe`. Следует обратить внимание на то, что при использовании префикса «`\\?\`» необходимо указывать только абсолютный путь к файлу, относительные пути не допускаются.

2.4.4 Специальные файлы

В продолжение концепции унификации ввода–вывода, принятой в UNIX, в качестве имен файлов в Windows также могут использоваться имена основных символьных устройств компьютера. Например, такие как **PRN** – для принтера, **CON** – для консоли (клавиатура для ввода и дисплей для вывода), **COM1, 2, 3, 4** – для последовательных портов, **NULL** – для фиктивного устройства ввода–вывода и др. Эти устройства называют специальными файлами. Рассмотрим в качестве примера команду ввода текста с клавиатуры в файл **F1.txt**, находящийся в директории **work** на диске **d**:

```
copy con d:\work\F1.txt.
```

После ввода этой команды консоль приступит к выполнению копирования символов с устройства **CON** (клавиатура). Символы, соответствующие нажимаемым клавишам, будут записываться в файл **F1.txt**. Для того чтобы прекратить процесс копирования, необходимо передать специальный управляющий код **^Z** – «конец передачи». Этот код вводится при совместном нажатии клавиш **Ctrl** и **z** (**Ctrl+z**). Реализация примера приведена на рисунке 4.

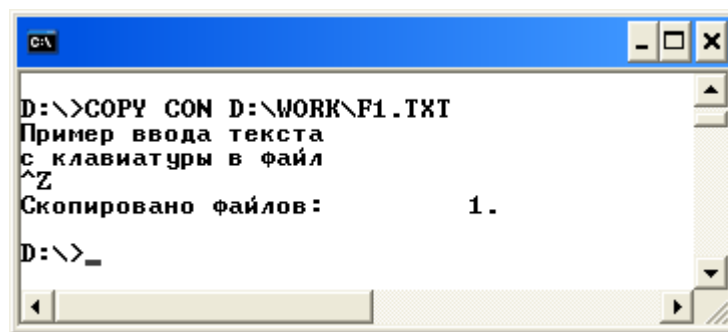


Рисунок 4 – Пример использования специального файла

2.5 Несколько команд в одной строке и условное выполнение

Можно объединять несколько команд в одной командной строке. Реализуется объединение с помощью операторов, описание которых приведено в таблице 1.

Таблица 1 – Символы объединения команд и описание их применения

Символы	Синтаксис	Описание применения
&	команда1 & команда2	Разделение размещенных в одной строке без-условно выполняемых команд. Выполняются команда1, команда2 и т. д.
&&	команда1 && команда2	Разделение условно выполняемых команд. Команда2 , стоящая справа от символов && , выполняется только если команда1 , стоящая слева, была выполнена успешно
 	команда1 команда2	Разделение условно выполняемых команд. Команда2 , стоящая справа от символов , выполняется только если команда1 , стоящая слева, не была выполнена успешно (код завершения 1 или больше)
()	(команда1 & команда2)	Используется для группировки команд
; или ,	команда1 пар1;пар2	Используется для разделения параметров команды
Примечание – Если символы «амперсанд» & , «вертикальная черта» и скобки () используются не специальным образом, а как символы в составе аргументов, то им должен предшествовать управляющий символ (^) или кавычки.		

Возможность исполнения следующей в строке команды может зависеть от успешности выполнения предыдущей команды. Поэтому говорят об условном либо безусловном выполнении команд. Условия выполнения определяются символами, используемыми для разделения команд. Будет выполняться команда, стоящая справа от символа условной обработки, или нет – зависит от успешности выполнения команды, стоящей слева. Считается, что команда выполнена успешно, если она возвращает код завершения 0 или не возвращает никакого кода.

Оператор условного выполнения действует только на ближайшую команду. Рассмотрим пример:

```
xcopy d:\work\fl.txt d:\ffl.txt && type d:\ffl.txt & help type
```

Здесь команда **type** будет выполняться только при успешном выполнении команды **xcopy**. Команда **help** будет выполнена в любом случае.

Использование скобок позволяет изменить указанный порядок:

```
xcopy d:\work\fl.txt d:\ffl.txt &&(type d:\ffl.txt & help type)
```

Теперь обе команды, **type** и **help**, будут выполнены только при успешном выполнении **xcopy** и обе не будут выполняться, если **xcopy** выполнится с ошибкой.

2.6 *Использование переменных среды в Cmd.exe*

2.6.1 *Блок среды командной оболочки*

Блок среды (environment block) командной оболочки **Cmd.exe** содержит переменные, используемые для управления поведением сеанса консоли и операционной системы в целом. Выделяют два типа переменных среды: системные и локальные.

Системные переменные среды определяют поведение глобальной среды операционной системы. Системные переменные среды заданы заранее в операционной системе и доступны для всех процессов Windows XP. Для изменения этих переменных требуются привилегии администратора.

Локальные переменные среды становятся доступны при входе в систему пользователя, для которого они были созданы. Таким образом, локальные переменные действуют только в сеансе текущего пользователя.

Все переменные можно отнести к нескольким категориям с разным уровнем приоритета. Далее представлен список категорий в порядке убывания приоритета:

- встроенные системные переменные среды;
- системные переменные, указанные в кусте **HKEY_LOCAL_MACHINE**;

- локальные переменные куста **HKEY_CURRENT_USER**;
- переменные среды и пути, указанные в файле **Autoexec.bat**;
- переменные среды и пути, указанные в сценарии входа в систему (если он имеется);
- переменные, используемые интерактивно в пакетном файле или сценарии.

2.6.2 Локализация изменений в блоке среды

Если в консоли ввести команду **Cmd.exe**, то будет запущен новый экземпляр командной строки (рисунок 5).

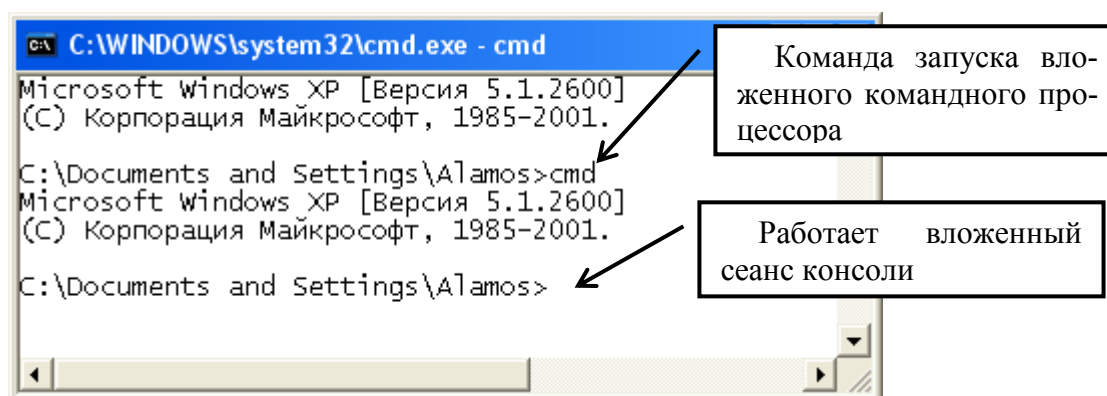


Рисунок 5 – Запуск вложенного сеанса командного процессора

Каждый новый экземпляр **Cmd.exe** наследует среду своего родителя. В блок среды вложенного сеанса **Cmd.exe** можно вносить изменения, которые не будут влиять на среду родительского процесса. Это позволяет сохранять исходную среду и возвращаться к ней после завершения дочернего сеанса командной оболочки. Завершить работу дочерней консоли можно, введя команду **exit**.

Есть другой способ локализовать действие изменений в блоке среды командного процессора. Он заключается в использовании команд **setlocal** и **endlocal**. Команда **setlocal** открывает вложенный локальный блок среды, а **endlocal** его удаляет. Любые изменения, сделанные в блоке среды после ввода команды **setlocal**, отменяются после ввода парной ей команды **endlocal**. С помощью этих команд можно создать до 32 вложенных локальных блоков окружения. Заметим, что это работает только в пакетном файле

2.6.3 Установка значений переменных среды

Для создания, удаления или отображения переменных среды используется команда **set**. Эта команда изменяет переменные только в среде текущего сеанса оболочки.

Некоторые варианты использования команды **set**:

- для отображения значения переменной:

set <имя_переменной> (пример – **set Temp**);

– для добавления переменной:

set <имя_переменной>=<значение> (пример – **set Temp=C:\temp**);

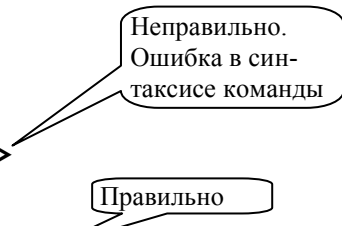
– для удаления переменной:

set <имя_переменной>= (пример – **set Temp =**).

Допускается использование специальных символов в качестве значений переменных. В качестве значения переменной принимаются все символы, следующие после знака равенства. При использовании специальных символов, таких как <, >, & или ^, перед ними следует помещать управляющий символ (^). Это отменяет специальное действие символа, и он просто вставляется в текст переменной. Используя кавычки, можно получить аналогичный результат, но кавычки будут также включены в значение переменной. Вышесказанное иллюстрируется примером, в котором назначается новый вид приглашения. Чтобы заставить консоль выводить приглашение в новом виде (таком, например, «**Новое приглашение**>»), необходимо изменить значение переменной **prompt** блока среды. Обратите внимание на символ > в тексте нового приглашения. Это специальный символ, используемый для перенаправления вывода. Чтобы включить этот символ в текст параметра команды, необходимо отменить его специальное использование. Далее приведена копия строк консоли с командами, реализующими несколько вариантов поставленной задачи.

Пример ввода специального символа

```
1 Microsoft Windows XP [Версия 5.1.2600]
2 (C) Корпорация Майкрософт, 1985-2001.
3
4 C:\Documents and Settings\Alamos>d:
5
6 D:\>set prompt
7 PROMPT=$P$G
8
9 D:\>set promt=Новое приглашение>
10
11
12 D:\>set prompt=Новое приглашение^>
13
14 Новое приглашение>set prompt
15 PROMPT=Новое приглашение>
16
17 Новое приглашение>set prompt=Новое приглашение">"
18
19 Новое приглашение">"set prompt
20 PROMPT=Новое приглашение">"
21
22 Новое приглашение">"set prompt="Новое приглашение">"
23
```



```

24 "Новое приглашение>"set prompt
25 PROMPT="Новое приглашение>"
26
27 "Новое приглашение>"set prompt=$p$g
28
29 D:\>

```

Чтобы задать включающее специальный символ значение «Новое приглашение>», вводим команду **set** в следующем виде:

```
set prompt=Новое приглашение^>
```

Для присвоения переменной **prompt** значения «"Новое приглашение>"», содержащего в своем составе кавычки, вводим команду:

```
set prompt="Новое приглашение>"
```

При обработке имен переменных регистр символов не учитывается. Однако команда **set** отображает значение переменной так, как оно было введено. Можно комбинировать символы нижнего и верхнего регистра в именах переменных, чтобы облегчить восприятие кода (например, **UserName**).

Примечание. Максимальный общий размер всех переменных среды, включая имена переменных и знак равенства, составляет 65 536 байт. При этом максимальный размер одной переменной в блоке окружения составляет 8 192 байта.

2.6.4 Обращение к значениям переменных среды

Если в командной строке используется имя переменной блока среды, обрамленное знаками процента (например, **%prompt%**), то оно интерпретируется командным процессором **Cmd.exe** как значение этой переменной. В приведенном примере командный процессор перед выполнением команды выполнит поиск всех вхождений **%prompt%** и замену их на значение этой переменной.

Пусть требуется вывести на экран значение переменной **prompt**. Выше рассматривалась возможность вывода значения переменной среды с помощью команды **set**:

```

D:\>set prompt
PROMPT=$P$G

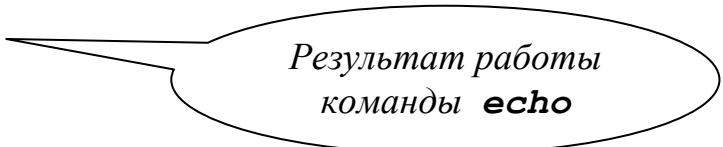
```

Мы видим, что в этом случае выводятся имя и значение переменной, разделенные символом «=». Воспользовавшись командой **Echo**, можно вывести только значение переменной:

```

D:\>echo %prompt%
$P$G
D:\>

```



Результат работы команды **echo**

2.6.5 Переменные среды для Windows XP

В таблицах 2 и 3 приведен список локальных (для пользователя) и системных переменных среды Windows XP.

Таблица 2 – Некоторые системные переменные среды Windows XP

Переменная	Значение
CMDEXTVERSION	Номер версии текущих расширений командного процессора
COMPUTERNAME	Имя компьютера
COMSPEC	Путь к исполняемой командной оболочке
DATE	Текущая дата. Использует тот же формат, что и команда <code>date /t</code>
ERRORLEVEL	Код завершения последней использовавшейся команды
HOMEDRIVE	Имя диска локальной рабочей станции, связанного с основным каталогом пользователя
HOMEPATH	Полный путь к основному каталогу пользователя. Основной каталог пользователя указывается в оснастке «Локальные пользователи и группы»
HOMESHARE	Сетевой путь к общему основному каталогу пользователя
NUMBER_OF_PROCESSORS	Количество установленных процессоров
OS	Имя операционной системы
PATH	Пути поиска для исполняемых файлов
PATHEXT	Список расширений файлов, которые рассматриваются операционной системой как исполняемые
PROCESSOR_ARCHITECTURE	Тип архитектуры процессора. Значения: <code>x86</code> , <code>IA64</code>
PROCESSOR_IDENTIFIER	Описание процессора
PROCESSOR_LEVEL	Номер модели процессора
PROCESSOR_REVISION	Номер модификации процессора
PROMPT	Параметры запроса командной строки для текущего интерпретатора. Создается командой <code>Cmd.exe</code>
RANDOM	Произвольное десятичное число от 0 до 32767. Создается командой <code>Cmd.exe</code>
SYSTEMDRIVE	Имя диска, содержащего корневой каталог Windows XP
SYSTEMROOT	Размещение системного каталога Windows XP
TEMP и TMP	Возвращает временные папки, по умолчанию используемые приложениями, которые доступны пользователям, выполнившим вход в систему. Некоторые приложения требуют переменную <code>TEMP</code> , другие – переменную <code>TMP</code>
TIME	Текущее время. Использует тот же формат, что и команда <code>time /t</code> . Создается командой <code>Cmd.exe</code>
WINDIR	Размещение каталога операционной системы

Таблица 3 – Некоторые локальные переменные среды Windows XP

Переменная	Значение
ALLUSERSPROFILE	Размещение профиля «All Users»
APPDATA	Используемое по умолчанию размещение данных приложений
CD	Путь к текущей папке
CMDCMDLINE	Строка команды, с помощью которой был запущен данный экземпляр Cmd.exe
LOGONSERVER	Имя контроллера домена, который проверял подлинность текущего сеанса
PROMPT	Параметры запроса в командной строке для текущего сеанса
TEMP и TMP	Временные папки для временных файлов, по умолчанию используемые приложениями
USERDOMAIN	Имя домена, содержащего список учетных записей пользователей
USERNAME	Имя пользователя, выполнившего вход в систему
USERPROFILE	Возвращает размещение профиля для текущего пользователя

2.7 Операторы перенаправления ввода–вывода команд

Операторы перенаправления ввода–вывода обеспечивают замену стандартных потоков ввода–вывода, используемых командами по умолчанию, на другие потоки или файлы. Ссылка на заменяющий поток ввода–вывода осуществляется путем указания дескриптора потока или дескриптора открытого файла до или после символа перенаправления. Стандартными являются потоки ввода с устройства «консоль» – **CON** и вывода на устройство «консоль» – **CON**. Ниже в таблице 4 представлены дескрипторы стандартных и других потоков, доступные приложению по умолчанию.

Таблица 4 – Дескрипторы стандартных потоков

Обозначение потока	Значение дескриптора	Описание потока
STDIN	0	Стандартный ввод с устройства CON – клавиатуры
STDOUT	1	Стандартный вывод на устройство CON – окно консоли
STDERR	2	Поток вывода сообщений об ошибках в окно консоли

Далее в таблице 5 приводится описание символов, используемых для обозначения операторов перенаправления потоков ввода и вывода команд.

Таблица 5 – Операторы перенаправления ввода – вывода

Оператор перенаправления	Описание
>	Записывает данные стандартного вывода команды в файл или на устройство, например, на принтер (PRN)
<	Читает вместо стандартного потока ввода данные из файла или устройства, например, из последовательного канала (COM1)
>>	Добавляет данные стандартного вывода команды в конец файла, не удаляя при этом существующей информации из файла
D1 >& D2	Направляет данные с выхода одного дескриптора (D1) на вход другого дескриптора (D2)
D1 <& D2	Считывает входные данные первого дескриптора (D1) с выхода второго дескриптора (D2)
cmd1 cmd2	Направляет данные стандартного вывода одной команды (cmd1) в поток стандартного ввода другой команды (cmd2). Такое объединение команд называют конвейером, а механизм обмена данными носит название «канал» (pipe)

После ввода оператора **<** или **>** необходимо указать, откуда читать и куда записывать данные. Можно задать имя файла или любой из существующих стандартных дескрипторов. Если дескриптор не указан, то по умолчанию оператором перенаправления ввода **<** будет использоваться ноль (0) – поток **STDIN**, а оператором перенаправления вывода **>** будет использована единица (1) – поток **STDOUT**. Для задания произвольного номера дескриптора перед ним вводится символ **&**. Например, для перенаправления вывода сообщений об ошибках из потока **STDERR** в стандартный поток вывода необходимо указать **<&2**.

Если осуществляется замена одного дескриптора на другой, то такая процедура называется дублированием дескриптора. В качестве оператора дублирования используется символ **&** (амперсанд). При дублировании дескриптора происходит копирование всех его исходных характеристик.

Рассмотрим в качестве примера перенаправление вывода команды **dir** в файл **File.txt**. Для этого можно записать команду **dir** в следующем виде:

```
dir > d:\work\file.txt
```

При этом сообщения об ошибках будут отправляться в поток **STDERR** (поток со значением дескриптора – 2). Этот поток по умолчанию осуществляет вывод информации в окно консоли. Чтобы обеспечить запись информации об ошибках в файл, необходимо перенаправить вывод потока 2 также в файл и

указать символ **>>**, обеспечивающий в этом случае добавление данных, а не перезапись файла:

```
dir > d:\work\file.txt "&"2>>d:\work\file.txt /ggg
```

В примере запись **/ggg** является недопустимым ключом и используется для искусственного инициирования вывода информации об ошибке. Этот прием позволит наблюдать добавление информации об ошибке в конец указанного файла.

2.8 Команды-фильтры и конвейер команд

Фильтрами называют команды, которые используют в качестве ввода стандартный поток **STDIN**, а в качестве вывода – стандартный поток **STDOUT**, т. е. ввод с клавиатуры и вывод в окно консоли. Такие команды удобно объединять в цепочки с помощью оператора канала «**|**» и формировать таким образом то, что называется конвейером команд. Другое наименование – канал (pipe). Оператор канала направляет во временный файл стандартный вывод команды, стоящей слева от оператора. Затем информация из временного файла направляется на стандартный вход команды, стоящей справа от оператора.

К фильтрам относятся команды **MORE**, **SORT**, **FIND**. Фильтры делят, упорядочивают или выделяют отдельные части из проходящей через них информации. Приведем в качестве примера следующую цепочку команд:

```
dir d:\ | find "txt" | sort /R | more
```

В этой цепочке информация о корневой директории диска **D**, выдаваемая командой **dir**, направляется на вход команды **find**, которая выделяет из входного потока строки, содержащие подстроку **"txt"** и направляет свой вывод на вход команды **sort**. Эта команда упорядочивает поток входящих строк в порядке, обратном алфавитному, и выдает их на вход команды **more**. Последняя выводит поток строк на экран порциями, соответствующими размеру окна консоли.

Конвейеризацию команд можно объединять с перенаправлением ввода–вывода. Например, можно рассмотренную выше цепочку команд заставить сохранять результаты в файле **file.txt**:

```
dir d:\ | find "txt" | sort /R | more >d:\file2.txt
```

В этом примере символы из потока стандартного вывода команды **more** перенаправляются в текстовый файл: **more >d:\file2.txt**.

3 *Выполнение работы*

3.1 *Подготовка к занятию*

При подготовке к занятию необходимо изучить теоретический материал, законспектировать назначение и синтаксис команд **DIR**, **MD**, **CD**, **RD**, **COPY**, **DEL**, **TYPE**, **EDIT**, **HELP**. Используйте теоретический раздел данного пособия, справочную информацию, получаемую по команде **HELP** в консоли, справочник параметров командной строки [1], работы В. Э. Фигурнова [2], источники в Интернет и др.

3.2 *Запуск и настройка окна консоли*

Цель: научиться запускать и настраивать командную оболочку.

Создание консольного окна Windows. Для этого в Windows XP выберите и выполните один из двух вариантов действий:

- откройте меню **Пуск -> Программы -> Стандартные** и выберите **«Командная строка»**;
- откройте окно **Выполнить** в меню **Пуск**, введите команду **cmd** и нажмите кнопку **«Выполнить»**.

Настройка окна консоли. Щелкните по значку системного меню в левом верхнем углу окна консоли и выберите команду **«Свойства»**. Попробуйте изменить свойства консольного окна, как указано ниже:

а) на странице свойств **«Общие»**:

- настройте стек команд. В области **«Запоминание команд»** выберите или введите значение (от 1 до 999) в поле **«Размер буфера»**. Затем выберите или введите значение (от 1 до 10) в поле **«Количество буферов»**;
- настройте режим копирования и вставки. В области **«Редактирование»** установите флажки **«Выделение мышью»** и **«Быстрая вставка»**;

б) на странице свойств **«Расположение»**:

- настройте размер буфера консоли. В области **«Размер буфера экрана»** оставьте значение 80 в поле **«Ширина»** и введите или выберите значение в диапазоне 200–2500 в поле **«Высота»**;
- настройте положение и размер окна консоли. В области **«Размер окна»** задайте значение параметра **«Высота»**. В области **«Размер окна»** задайте значение параметра **«Ширина»**. Снимите флажок **«Автоматический выбор»** и в области **«Положение окна»** задайте значения полей **«Левый край»** и **«Верхний край»**;

в) на странице свойств «Цвета» установите:

- цвет текста на экране черный (0,0,0);
- цвет фона экрана белый (255, 255, 255).

Завершив выполнение настроек, нажмите кнопку «ОК». В диалоговом окне «Изменение свойств» выберите вариант «Сохранить свойства для других окон с тем же именем».

3.3 Знакомство с отдельными командами

Цель: получить навыки использования консоли и команд консольного режима для управления работой системы.

3.3.1 Просмотр директорий и свойств файлов. Команда DIR

Применить команду **DIR**:

- для просмотра корневой директории диска **D**;
- для просмотра своей директории **NNNN_XX** (где **NNNN_XX** – номер вашей зачетной книжки, например **9032_01**). Если указанной директории нет, ее нужно предварительно создать;
- для просмотра информации об отдельных файлах.

Попробовать выполнить команду с использованием различных опций (ключей).

Контроль выполнения – демонстрация отдельных заданий по требованию преподавателя.

3.3.2 Создание директорий

Используя команды **MD**, **CD** и др., создать на диске **D** в директории **NNNN_XX** дерево директорий, представленное на рисунке 6 (9032_1 заменить на номер вашей зачетной книжки). Все вводимые в процессе выполнения команды записать в рабочую тетрадь (отчет).

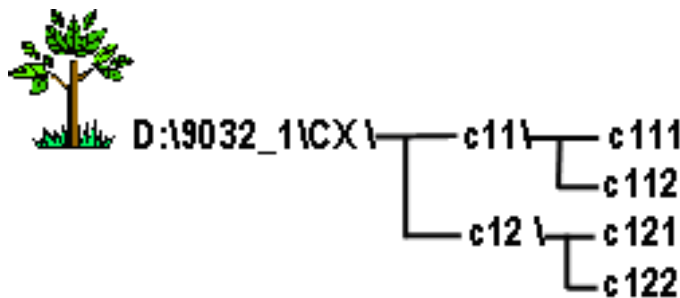


Рисунок 6 – Дерево директорий

Контролировать правильность выполнения с помощью команды **DIR**. Результаты, выводимые командой **Dir**, последовательно сохранять в файле **Tree_result.txt** в рабочей директории (использовать перенаправление вывода).

3.3.3 Копирование и просмотр файлов

Рассмотреть различные варианты использования команды **COPY**:

а) использовать команду (**copy con <имя файла>**) для создания в директории **c11** файла **F11.ttt** путем копирования символов, вводимых с консоли. Ввести 3 строки:

- 1) свои **имя и фамилию**;
- 2) строку **"Текст введен с консоли"**;
- 3) **дату и время**.

Завершить копирование посылкой кода **"Конец передачи"** – **^Z** (клавиша **Ctrl** + клавиша **z**);

б) использовать команду **copy** для создания копии файла **F11.ttt** в директории **c122** под именем **F11c.ttt**;

в) с помощью команды **copy** вывести текст файлов **F11.ttt** и **F11c.ttt** на экран;

г) выполнить просмотр содержимого файлов **F11.ttt** и **F11c.ttt** с помощью команды **TYPE**;

д) просмотреть содержимое файлов **F11.ttt** и **F11c.ttt** с помощью редактора **EDIT**;

е) рассмотреть созданное дерево директорий с помощью программы «Проводник» и убедиться в том, что задание выполнено правильно. Можно использовать команду **TREE**;

ж) предъявить преподавателю полученные результаты и записи. Используя команды **DEL** и **RD**, удалить созданные файлы и директории.

3.3.4 Перенаправление ввода–вывода

Составить строку команд, которая будет из информации о директории, получаемой с помощью команды **Dir**, отбирать строки с именами файлов заданного типа и записывать в текстовый файл.

Сохранить в отдельном файле справочную информацию по командам, указанным в пункте 3.1. Использовать команду **HELP команда /?** и перенаправление вывода.

3.3.5 Условное выполнение команд в строке

С помощью команды **Copy** вывести содержимое созданного в предыдущем пункте файла на экран. Если команда выполнится с ошибкой, то выпол-

нить команду **dir** для директории, указанной в команде копирования, и справку по этой команде.

3.3.6 Использование значений переменных среды

Составьте и выполните команды, которые запишут в текстовый файл пути доступа к временным файлам, текущее значение переменной **ERRORLEVEL**, имя пользователя, путь в системную директорию, имя пользователя. Для вывода используйте команду **ЕCHO** и значения переменных среды.

Работа завершается оформлением отчета и защитой.

4 Вопросы для самопроверки

- 1 Пояснить понятие «командная строка»(консоль).
- 2 Из каких элементов состоит команда в режиме консоли?
- 3 Какие элементы включает спецификация файла?
- 4 Пояснить понятие текущей (рабочей) директории.
- 5 Пояснить понятия абсолютного и относительного пути.
- 6 Чем отличаются записи абсолютного и относительного путей?
- 7 Перечислите имена специальных файлов в DOS и Windows.
- 8 Перечислить операторы перенаправления ввода–вывода и привести примеры их применения.
- 9 Какие программы называют фильтрами? Привести примеры использования фильтров.
- 10 Что такое конвейер команд и как его организовать?
- 11 Из каких элементов состоит команда в консольном режиме?
 - а) имя команды, операнды, разделители;
 - б) разделители, операнды, ключи, имя команды;
 - в) операнды, ключи, имя команды.
- 12 Что дает использование понятия «текущая директория»?
 - а) сокращает запись команды;
 - б) ускоряет поиск файла;
 - в) уменьшает время нахождения адреса файла на диске;
 - г) увеличивает скорость считывания данных файла;
 - д) уменьшает время доступа к данным.

13 Какие высказывания верно определяют отличительные признаки записи абсолютного и относительного путей?

- а) в начале записи абсолютного пути указано устройство;
- б) в начале записи абсолютного пути указана корневая директория диска;
- в) в начале записи абсолютного пути указана текущая директория;
- г) в начале записи относительного пути указана текущая директория;
- д) в начале записи относительного пути не указана корневая директория диска;
- е) в начале записи абсолютного пути указан символ «\»;
- ж) в начале записи относительного пути не указан символ «\».

14 Что представляет собой маршрут доступа к файлу (Path)?

- а) перечень каталогов, которые система последовательно должна прочитать для нахождения адреса начала файла на диске;
- б) перечень каталогов, которые прикладная программа последовательно должна прочитать для нахождения адреса начала файла на диске;
- в) перечень кластеров, которые система должна прочитать для нахождения файла на диске;
- г) инструкцию оператору для открытия файла нужным текстовым документом.

Литература

1 Станек, Уильям Р. Командная строка Microsoft Windows : справочник администратора / Уильям Р. Станек; пер. с англ. – М. : Издательско-торговый дом «Русская Редакция», 2004. – 480 с.

2 Фигурнов, В. Э. IBM PC для пользователя / В. Э. Фигурнов. – 2-е изд. – М. : Финансы и статистика, 1991 (и более поздние издания).

3 Дибров, А. Особенности реализации командного процессора cmd.exe операционных систем WinNT [Электронный ресурс] . – 2009. – Режим доступа: <http://www.rsdn.ru/article/winshell/NTCommandProcessor.xml>. – Дата доступа: 20.10.2010.

4 Bat файлы [Электронный ресурс]. – 2001. – Режим доступа: www.firststeps.ru/msoffice/win/bats/bats1.html. – Дата доступа: 20.10.2010.

5 Имя файла [Электронный ресурс]. – 2010. – Режим доступа: http://ru.wikipedia.org/wiki/Имя_файла. – Дата доступа: 20.10.2010.

Учебное издание

Москалев Александр Анатольевич
Криштапович Наталия Иосифовна

АРХИТЕКТУРА ОПЕРАЦИОННЫХ СИСТЕМ

Лабораторный практикум
для слушателей переподготовки по специальности 1-40 01 73
«Программное обеспечение информационных систем»
всех форм обучения

В 4-х частях

Часть 1

Интерфейс пользователя в консольном режиме

Редактор Г. С. Корбут
Корректор А. В. Тюхай

Подписано в печать 10.06.2011.
Гарнитура «Таймс».
Уч.-изд. л. 1,3.

Формат 60x84 1/16.
Отпечатано на ризографе.
Тираж 100 экз.

Бумага офсетная.
Усл. печ. л. 1,51.
Заказ 129.

Издатель и полиграфическое исполнение: учреждение образования
«Белорусский государственный университет информатики и радиоэлектроники»
ЛИИ №02330/0494371 от 16.03.2009. ЛП №02330/0494175 от 03.04.2009.
220013, Минск, П. Бровки, 6

